Persistencia: DAO y Singleton

El problema

Todo lo que un programa construye vive en memoria, y la memoria se libera cuando el programa termina. Una partida ganada, el mejor puntaje de la semana, el historial de batallas, todo desaparece al cerrar la ventana. Persistir es hacer que los datos sobrevivan a la ejecución, guardándolos en algún lugar que no dependa del proceso, y volver a construir los objetos a partir de ellos la próxima vez.

Las dos opciones de siempre son un archivo y una base de datos. Un archivo alcanza para poco volumen y una sola forma de leerlo. Una base de datos agrega lo que un archivo no tiene, consultas por cualquier criterio, ordenar y filtrar sin cargar todo en memoria, integridad de los datos y varios programas usando lo mismo a la vez. Para un proyecto de escritorio la base más simple es SQLite, que guarda toda la base en un único archivo, no necesita servidor ni instalación, y se usa desde Java con la misma biblioteca que cualquier otra base, JDBC.

Esta pre clase tiene dos partes. La primera es la técnica, cómo se conecta un programa Java a SQLite y cómo se leen y escriben filas. La segunda es la de diseño, dónde va ese código para que el modelo no dependa de la base, y es la que usa dos patrones. DAO, para separar el acceso a datos del resto del programa, y Singleton, para la conexión.

Persistir es guardar afuera del proceso y reconstruir los objetos después. El problema de diseño es que el modelo no se entere de cómo.

Preparación

Java trae JDBC, la interfaz para hablar con bases de datos, pero no trae el driver de cada base. El de SQLite es un archivo *sqlite-jdbc-3.50.3.0.jar*, que se descarga de https://github.com/xerial/sqlite-jdbc/releases y se copia a la carpeta *lib* del proyecto. En Visual Studio Code, los proyectos de la cátedra ya tienen esa carpeta como biblioteca referenciada, así que alcanza con copiar el archivo y esperar unos segundos a que el editor lo tome. Si el proyecto es nuevo, en la vista “Java Projects” se agrega desde “Referenced Libraries”.

Para comprobar que quedó bien, este programa crea el archivo *prueba.db* en la carpeta del proyecto y no tiene que imprimir ningún error:

```
import java.sql.Connection;
import java.sql.DriverManager;

public class Prueba {
    public static void main(String[] args) throws Exception {
        try (Connection conexion = DriverManager.getConnection("jdbc:sqlite:prueba.db")) {
            System.out.println("Conectado a " + conexion.getMetaData().getURL());
        }
    }
}
```

Si aparece “No suitable driver found”, el jar no está en el classpath. Si el editor lo tomó pero el programa no lo encuentra, cerrar y volver a abrir la carpeta suele resolverlo. Con Java 22 o más nuevo aparecen además unos avisos sobre acceso nativo al cargar el driver. Son avisos, no errores, y el programa sigue.

JDBC en cuatro clases

JDBC son unas pocas interfaces del paquete *java.sql*, y con cuatro alcanza para todo lo que hace falta en el proyecto. *Connection* es la conexión abierta con la base, y se obtiene con *DriverManager.getConnection()* y una URL que dice qué base y dónde. *PreparedStatement* es una sentencia SQL con huecos, los signos de pregunta, que se llenan con *setString()*, *setInt()* y parecidos antes de ejecutarla. *ResultSet* es el resultado de una consulta, un cursor que avanza fila por fila con *next()* y entrega cada columna con *getString()* o *getInt()*. Y *SQLException* es lo que cualquiera de las tres lanza cuando algo falla.

Las tres primeras son recursos que hay que cerrar, y la forma de no olvidarse es declararlas dentro de un *try* con recursos, que las cierra solo al salir del bloque, con o sin error:

```
String sql = "INSERT INTO puntajes (jugador, puntos, fecha) VALUES (?, ?, ?)";
try (PreparedStatement sentencia = conexion.prepareStatement(sql)) {
    sentencia.setString(1, "Ana");
    sentencia.setInt(2, 1250);
    sentencia.setString(3, "2026-10-07");
    sentencia.executeUpdate();
}

String consulta = "SELECT jugador, puntos FROM puntajes ORDER BY puntos DESC LIMIT ?";
try (PreparedStatement sentencia = conexion.prepareStatement(consulta)) {
    sentencia.setInt(1, 10);
    try (ResultSet filas = sentencia.executeQuery()) {
        while (filas.next()) {
            String jugador = filas.getString("jugador");
            int puntos = filas.getInt("puntos");
        }
    }
}
```

Los huecos no son una comodidad, son la forma correcta de meter un dato en una sentencia. Si el nombre del jugador se pega con + dentro del texto SQL, un jugador que se llame *Ana'); DROP TABLE puntajes;* – borra la tabla. Con *setString()* el valor viaja aparte de la sentencia y la base lo trata como un dato, sea lo que sea. El SQL que hace falta para el proyecto es poco, *CREATE TABLE IF NOT EXISTS* al arrancar, *INSERT* para guardar, *SELECT* con *ORDER BY* y *LIMIT* para consultar, y *DELETE* si hace falta borrar.

Connection, PreparedStatement, ResultSet y SQLException. Los tres primeros se cierran con un try con recursos, y los datos van siempre en los huecos.

El problema del acceso a datos

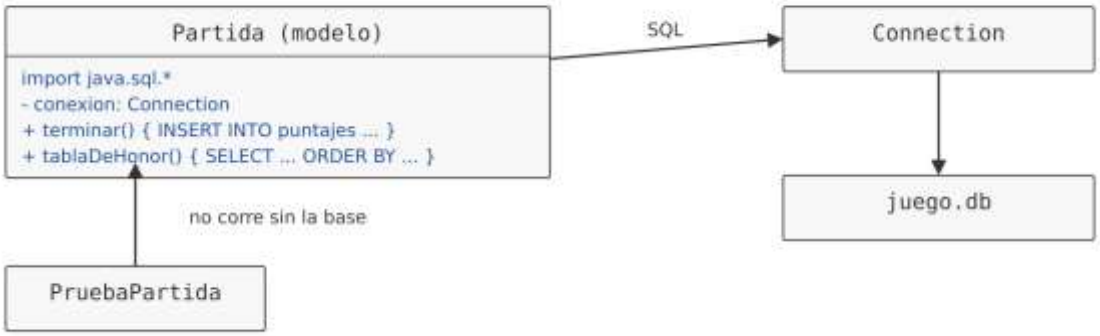
Con eso ya se puede guardar un puntaje. La primera versión, la más directa, escribe el SQL en el lugar donde la partida termina:

```java
public class Partida {

    private final Connection conexion;    // java.sql adentro del modelo

    public boolean terminar(String jugador) {
        String sql = "INSERT INTO puntajes (jugador, puntos, fecha) VALUES (?, ?, ?)";
        try (PreparedStatement sentencia = this.conexion.prepareStatement(sql)) {
            // ...
        }
    }
}
```

Funciona, y tiene el mismo defecto que una clase de dominio que importa *javax.swing*. El modelo, que hasta ahora no sabía que había una pantalla, ahora sabe que hay una base, cuál es y cómo se le habla. Los costos son tres. El modelo depende del motor, y cambiar SQLite por otra base o por un archivo es abrir el modelo. No se puede probar la partida sin una base de verdad, así que cada prueba escribe en el disco. Y el SQL de cada tabla queda repartido entre las clases que lo necesitan, con el *INSERT* en un lado y el *SELECT* en otro.



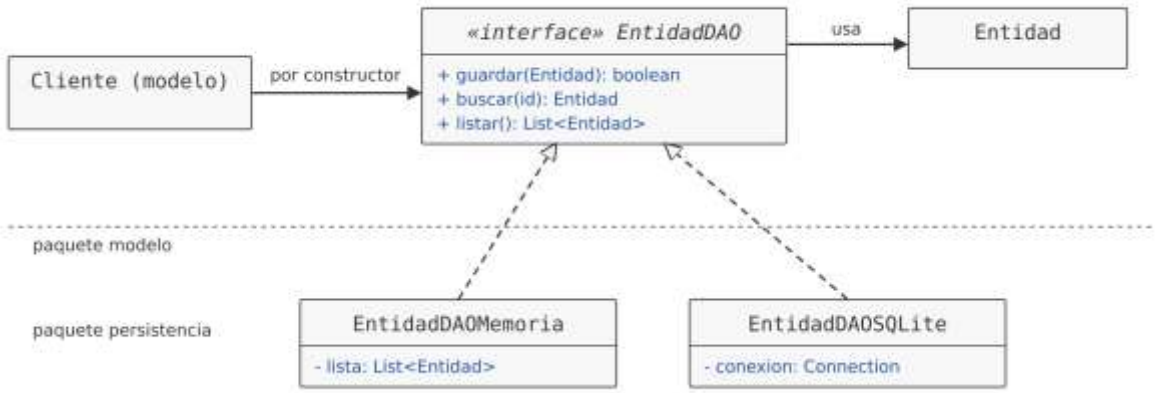
Sin el patrón. El modelo importa java.sql, conoce las tablas y no se puede probar sin la base.

DAO

La solución

Se separa el acceso a datos en objetos propios. Para cada entidad que se persiste se define una interfaz, el DAO, con las operaciones que el modelo necesita, guardar, buscar, listar, borrar, escritas en términos del dominio, con objetos y listas, sin una palabra de SQL. Esa interfaz vive en el paquete del modelo, porque la define la necesidad del modelo, igual que la interfaz del notificador o la del observador. Después se escribe una clase que la implementa con JDBC, en un paquete aparte, y es la única clase del programa que conoce la tabla. El modelo recibe el DAO por constructor y le pide las operaciones sin saber cómo se resuelven.

Los participantes son cuatro. La entidad, el objeto del dominio que se persiste. La interfaz DAO, con las operaciones. La implementación concreta, una por mecanismo de persistencia. Y el cliente, la clase del modelo que usa el DAO. Una segunda implementación en memoria, con una lista, sirve para las pruebas y para empezar a programar antes de tener la base.



Estructura de DAO. La interfaz vive en el modelo, la implementación con JDBC vive aparte, y el cliente no sabe cuál tiene.

El nombre viene de Data Access Object, y no es del catálogo de Gamma y compañía sino de otro, el de patrones de Java empresarial de 2001. Es una aplicación directa del principio de inversión de dependencias. La regla, el modelo, define la interfaz, y el detalle, la base, se adapta a ella.

Ejemplo

El juego guarda los mejores puntajes. La entidad es un *Puntaje*, con jugador, puntos y fecha, que nace válido o no nace:

```
public class Puntaje {

    private final String jugador;
    private final int puntos;
    private final LocalDate fecha;

    public Puntaje(String jugador, int puntos, LocalDate fecha) {
        if (jugador == null || jugador.isBlank() || puntos < 0 || fecha == null) {
            throw new IllegalArgumentException("Jugador, puntos y fecha son obligatorios");
        }
        this.jugador = jugador;
        this.puntos = puntos;
        this.fecha = fecha;
    }

    public String getJugador() { return this.jugador; }
    public int getPuntos() { return this.puntos; }
    public LocalDate getFecha() { return this.fecha; }
}
```

La interfaz declara lo que la partida necesita, y nada más. Está en el paquete *modelo*:

```java
public interface PuntajeDAO {
    boolean guardar(Puntaje puntaje);
    List<Puntaje> mejores(int cantidad);
}
```

La implementación con SQLite está en el paquete *persistencia*, recibe la conexión por constructor, crea la tabla si no existe, y traduce entre objetos y filas. Es la única clase que sabe que existe una tabla *puntajes*:

```java
public class PuntajeDAOSQLite implements PuntajeDAO {

    private final Connection conexion;

    public PuntajeDAOSQLite(Connection conexion) throws SQLException {
        if (conexion == null) {
            throw new IllegalArgumentException("La conexion es obligatoria");
        }
        this.conexion = conexion;
        try (Statement sentencia = conexion.createStatement()) {
            sentencia.execute("CREATE TABLE IF NOT EXISTS puntajes ("
                    + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
                    + "jugador TEXT NOT NULL, puntos INTEGER NOT NULL, fecha TEXT NOT
NULL)");
        }
    }

    @Override
    public boolean guardar(Puntaje puntaje) {
        String sql = "INSERT INTO puntajes (jugador, puntos, fecha) VALUES (?, ?, ?)";
        try (PreparedStatement sentencia = this.conexion.prepareStatement(sql)) {
            sentencia.setString(1, puntaje.getJugador());
            sentencia.setInt(2, puntaje.getPuntos());
            sentencia.setString(3, puntaje.getFecha().toString());
            return sentencia.executeUpdate() == 1;
        } catch (SQLException e) {
            return false;
        }
    }

    @Override
    public List<Puntaje> mejores(int cantidad) {
        String sql = "SELECT jugador, puntos, fecha FROM puntajes ORDER BY puntos DESC LIMIT
?";
        List<Puntaje> resultado = new ArrayList<>();
        try (PreparedStatement sentencia = this.conexion.prepareStatement(sql)) {
            sentencia.setInt(1, cantidad);
            try (ResultSet filas = sentencia.executeQuery()) {
                while (filas.next()) {
                    resultado.add(new Puntaje(filas.getString("jugador"),
filas.getInt("puntos"),
                            LocalDate.parse(filas.getString("fecha"))));
                }
            }
        } catch (SQLException e) {
            return resultado;
        }
        return resultado;
    }
}
```

El *catch* convierte la excepción de JDBC en el *false* o la lista vacía de la convención del curso, así el modelo no tiene que saber qué es una *SQLException*. La versión en memoria implementa la misma interfaz con una lista y un ordenamiento, y no tiene ni un *import* de *java.sql*:

```
public class PuntajeDAOMemoria implements PuntajeDAO {

    private final List<Puntaje> puntajes = new ArrayList<>();

    @Override
    public boolean guardar(Puntaje puntaje) {
        if (puntaje == null) {
            return false;
        }
        this.puntajes.add(puntaje);
        return true;
    }

    @Override
    public List<Puntaje> mejores(int cantidad) {
        List<Puntaje> ordenados = new ArrayList<>(this.puntajes);
        ordenados.sort((a, b) -> b.getPuntos() - a.getPuntos());
        return ordenados.subList(0, Math.min(cantidad, ordenados.size()));
    }
}
```

La partida recibe el DAO por constructor y lo usa en términos de puntajes:

```
public class Partida {

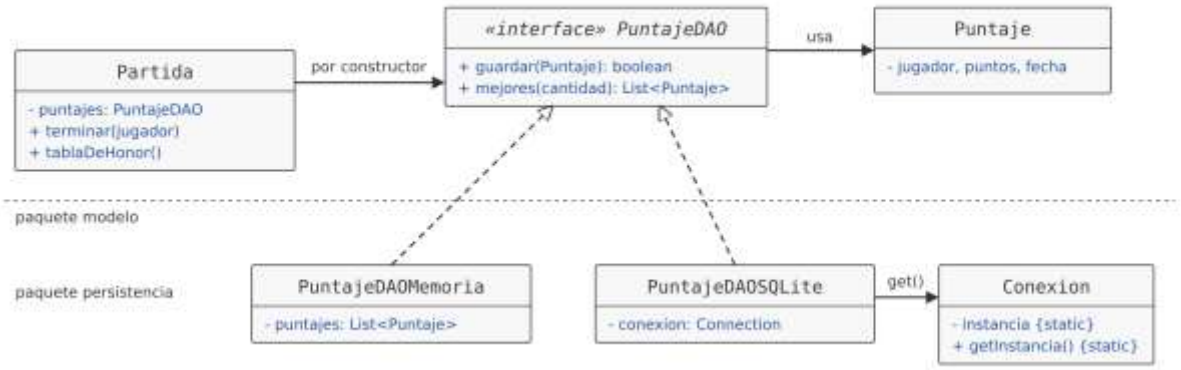
    private final PuntajeDAO puntajes;
    private int puntos;

    public Partida(PuntajeDAO puntajes) {
        if (puntajes == null) {
            throw new IllegalArgumentException("El DAO de puntajes es obligatorio");
        }
        this.puntajes = puntajes;
    }

    public boolean terminar(String jugador) {
        return this.puntajes.guardar(new Puntaje(jugador, this.puntos, LocalDate.now()));
    }

    public List<Puntaje> tablaDeHonor() {
        return this.puntajes.mejores(10);
    }
}
```

Partida no importa *java.sql*, y una prueba la construye con *new Partida(new PuntajeDAOMemoria())* sin tocar el disco. El programa real la construye con la versión SQLite.



El ejemplo con la estructura de DAO. Partida usa PuntajeDAO, y quien arma el programa decide si es la versión SQLite o la de memoria.

Ventajas y desventajas

- El modelo no conoce la base. Cambiar de motor, o pasar de un archivo a una base, es escribir otra implementación de la interfaz.
- El modelo se prueba sin base, con la implementación en memoria, y el SQL de cada tabla está en una sola clase.
- Es una capa más. Para cada entidad hay una interfaz y una o dos clases, y en un programa chico puede sentirse como mucho.
- La interfaz tiende a llenarse de consultas particulares, *mejoresDelMes()*, *porJugador()*, *entreFechas()*, y cada una es un método más en todas las implementaciones.

DAO. Una interfaz en el modelo con las operaciones en términos del dominio, y una implementación aparte que es la única que conoce la tabla.

Singleton para la conexión

Falta decidir de dónde sale la *Connection* que recibe el DAO. Abrir una conexión es costoso, y con SQLite abrir el mismo archivo dos veces desde el mismo programa es además un problema, porque las escrituras de una conexión bloquean a la otra. Hace falta una sola conexión para toda la ejecución, y que no haya forma de crear la segunda. Es el caso para el que existe Singleton, y el único de esta unidad:

```java
public class Conexion {

    private static Conexion instancia;
    private final Connection conexion;

    private Conexion() throws SQLException {
        this.conexion = DriverManager.getConnection("jdbc:sqlite:juego.db");
    }

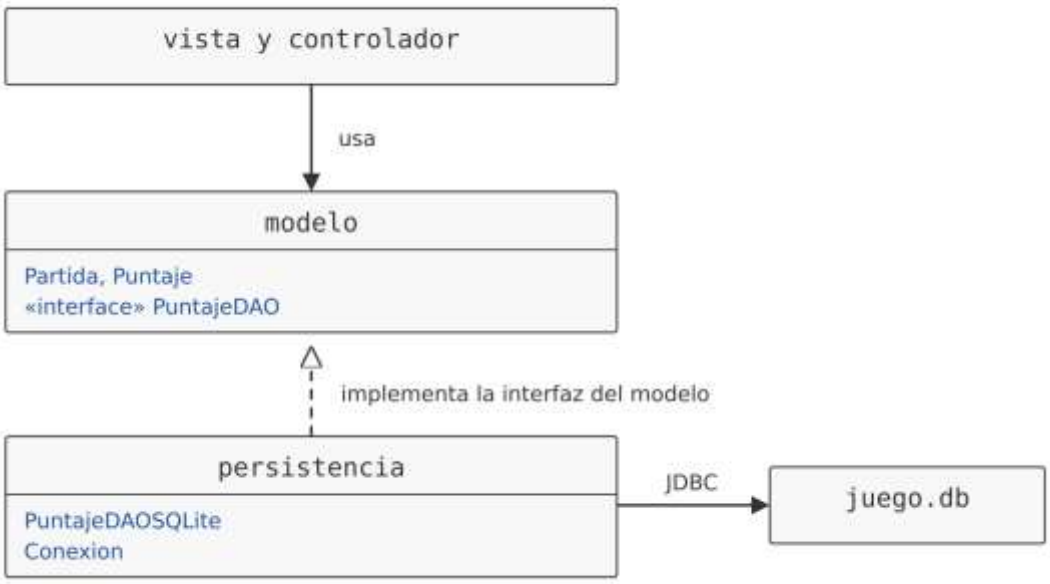
    public static Conexion getInstancia() throws SQLException {
        if (instancia == null) {
            instancia = new Conexion();
        }
        return instancia;
    }

    public Connection get() {
        return this.conexion;
    }
}
```

La regla del curso sigue igual. *getInstancia()* se llama una vez, en el programa principal, y la conexión viaja por constructor a los DAO. Ningún DAO llama a *Conexion.getInstancia()* por su cuenta, porque entonces no se podría construir con otra conexión, por ejemplo una base de prueba en memoria, que SQLite ofrece con la URL *jdbc:sqlite::memory:*. El programa principal queda así:

```java
Connection conexion = Conexion.getInstancia().get();
PuntajeDAO puntajes = new PuntajeDAOSQLite(conexion);
Partida partida = new Partida(puntajes);
```

Con todo junto, el programa queda en capas. La vista y el controlador arriba, el modelo en el medio con la partida, el puntaje y la interfaz del DAO, y la persistencia abajo con la implementación SQLite y la conexión. Las flechas de dependencia apuntan todas hacia el modelo, la persistencia depende del modelo y no al revés, y el archivo de la base queda afuera de todo.



Las capas del programa. La persistencia depende del modelo, porque implementa una interfaz que el modelo definió. El modelo no depende de nadie.

Singleton para la conexión, y nada más. Se obtiene una vez en el programa principal y viaja por constructor a los DAO.

El DAO genérico

Con dos o tres entidades aparece la repetición. *PuntajeDAO* y *PartidaDAO* tienen el mismo *guardar()*, el mismo *buscar()* y el mismo *listar()*, y las implementaciones con JDBC se parecen línea por línea, con solo la tabla y las columnas cambiando. Java tiene una herramienta para escribir una vez lo que solo cambia en el tipo, los genéricos, los mismos de *List*. Una interfaz *DAO* con *guardar(T)*, *buscar(int)* y *listar()* sirve para cualquier entidad, y una clase abstracta con JDBC puede implementar el recorrido del *ResultSet* dejando a cada subclase solo la traducción de una fila a un objeto. Es una lectura sugerida más que un requisito, y es el paso siguiente natural cuando el proyecto tiene más de dos tablas.

Para el proyecto

El proyecto pide guardar en SQLite el historial de partidas y los mejores puntajes, con DAO y Singleton. Los pasos son cuatro. Definir qué se guarda, una o dos entidades, con sus atributos. Escribir la interfaz DAO de cada una en el paquete del modelo, con las operaciones que la partida necesita y nada más. Implementarla primero en memoria, para que el juego funcione y se pruebe sin base, y después con SQLite. Y obtener la conexión con el Singleton en el programa principal, pasándola a los DAO por constructor. La pregunta para revisar el diseño es una sola, si alguna clase del paquete *modelo* importa *java.sql*, algo quedó en el lugar equivocado.

Bibliografía sugerida para profundizar

Oracle. *The Java Tutorials*, “JDBC Basics”.

- La referencia oficial de *Connection*, *PreparedStatement* y *ResultSet*, con los ejemplos de consultas con parámetros.

SQLite. *SQL As Understood By SQLite*. https://www.sqlite.org/lang.html

- La sintaxis de *CREATE TABLE*, *INSERT*, *SELECT* y *DELETE* tal como la acepta SQLite, con sus tipos de columna.

Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE Patterns: Best Practices and Design Strategies* (2.ª ed.). Prentice Hall.

- “Data Access Object”, en el capítulo de patrones de integración. Es la descripción original del patrón.

Martin, R. C. (2017). *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall.

- Capítulo 30, “The Database Is a Detail”. El argumento de por qué el modelo no debe depender de la base, en dos páginas.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

- “Table Data Gateway” y “Repository”, los dos patrones vecinos de DAO, para ver cómo se diferencian.